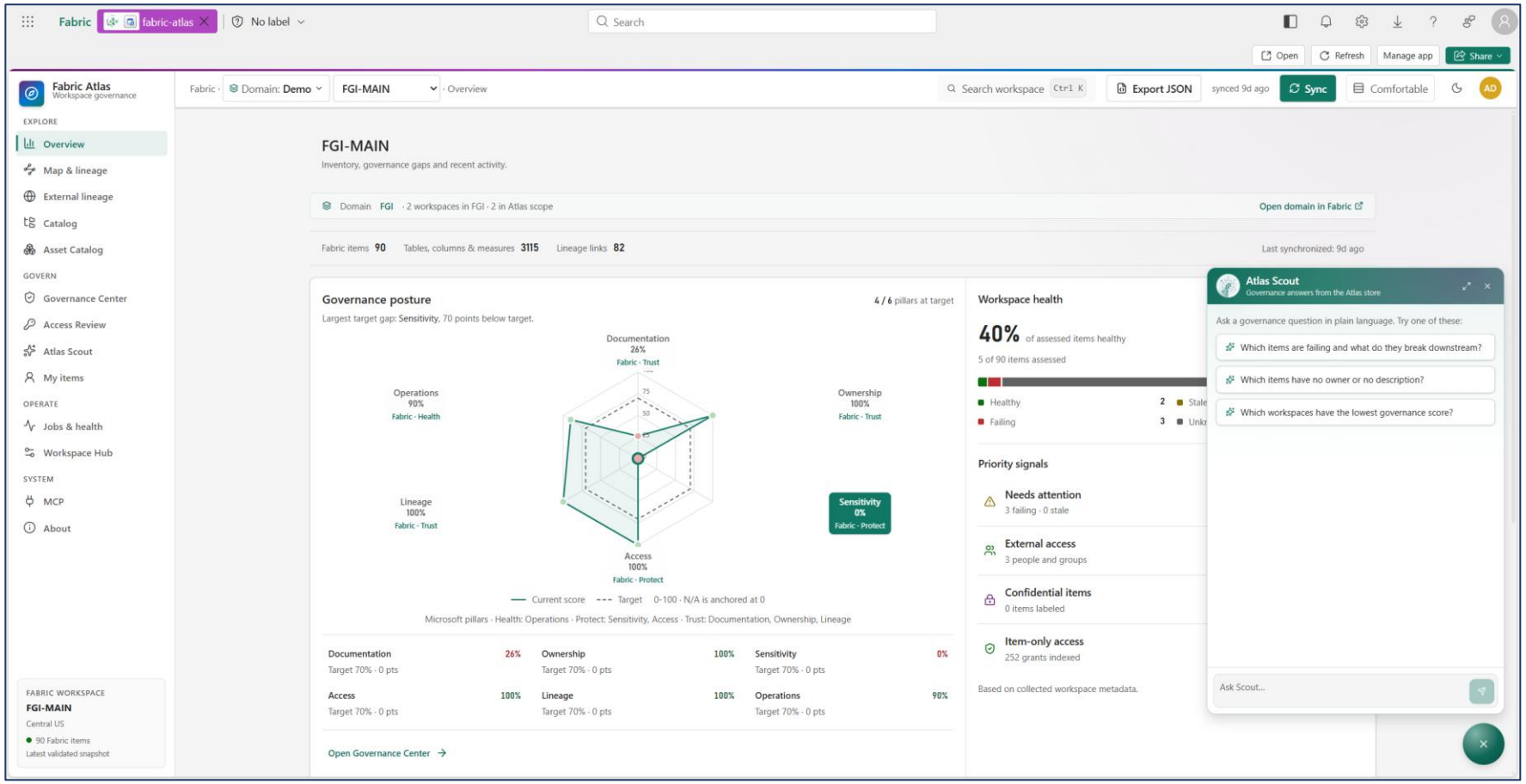
FABRIC ATLAS

Fabric governance, reimagined for the agentic era.

From catalog to Atlas Scout: how it was built

PRODUCT

A governance portal for Fabric Workspaces



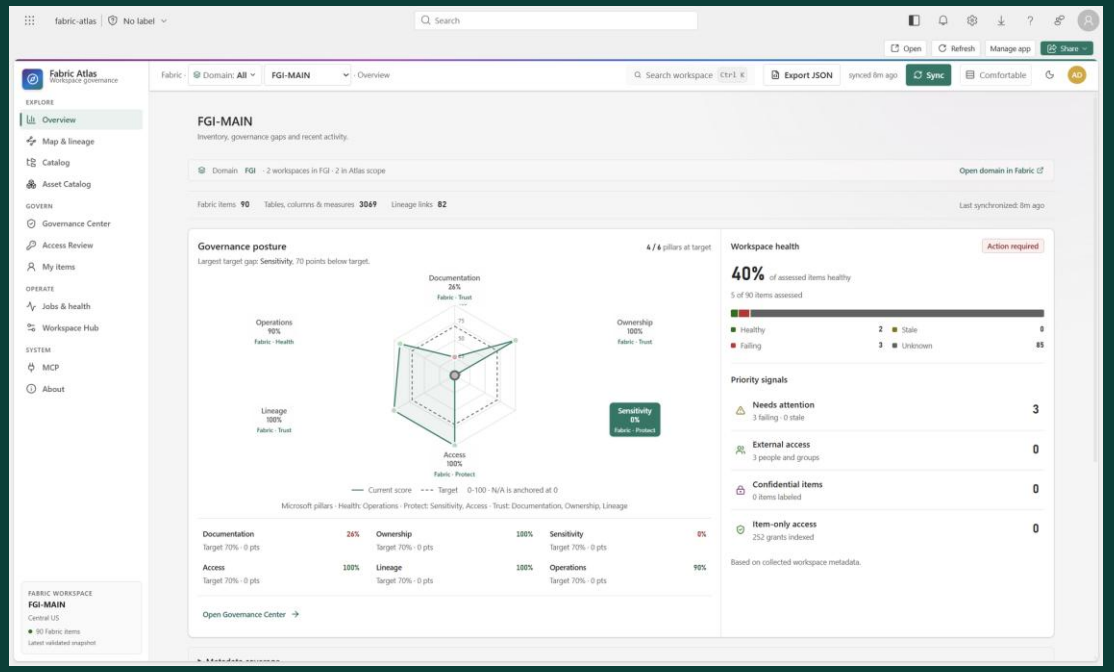
- Multi-workspace catalog, persistent sync status and switching.
- Catalog, lineage with OpenLineage, governance, connections and recent jobs.
- Semantic X-Ray, Departure Packs and Atlas Scout, a read-only Data Agent chat.
- Pages read the validated snapshot; they do not rescan Fabric.

Fabric Atlas 2.7.11 Overview with the Atlas Scout bubble; figures are live workspace evidence.

IMPLEMENTED SOLUTION OVERVIEW

What we built during the build window

From a single-workspace catalog to a governance portal with OpenLineage, connection governance and an agentic assistant, shipped as 59 releases from v1.0.0 to v2.9.1.



Ten weeks, eight milestones, one deployed product.

29 Aug	29–30 Aug	5–17 Sep	3 Oct	17–28 Oct	29 Oct	6–7 Nov	9 Nov
v1.0	**v1.5**	**v1.10–1.12**	**v2.0**	**v2.1–2.6**	**v2.7.0**	**v2.7.1–2.7.12**	**v2.8.0–2.9.1**
Map & lineage	**Governance**	**Operations**	**Rayfin-first**	**Depth**	**OpenLineage**	**Agentic**	**Connections**
Impact analysis, staged layout, item and object lineage	Governance Center, Access Review, Ctrl+K search, impact reports	Catalog table, exceptions, durable resumable sync	Multi-workspace scope, Rayfin Function collectors	Object lineage, Fix in Fabric, domains, facets, finding Why	Import, link to Fabric, External lane, NDJSON export	Atlas Scout, Sync Brief, identity sheet	Tenant inventory, owners, usage, policy readiness, departure pack

Build rhythm. Every release is tagged, documented in CHANGELOG.md and deployed with rayfin up. Patch releases ship verified fixes the same day.

Key achievements

59

tagged releases

v1.0.0 → v2.9.1, 292 commits

32

Rayfin entities

schema, Data API and typed client generated

1,986

automated tests

unit, component and contract tests, Node 24 CI

3

Scout entry points

page, bubble and Ask Scout, one conversation

2

upstream PRs

awesome-rayfin #207 and fabric-toolbox #659

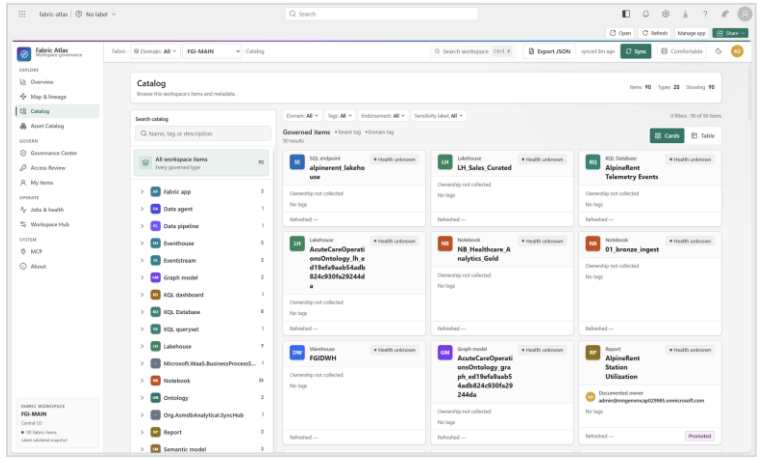
50

page whitepaper

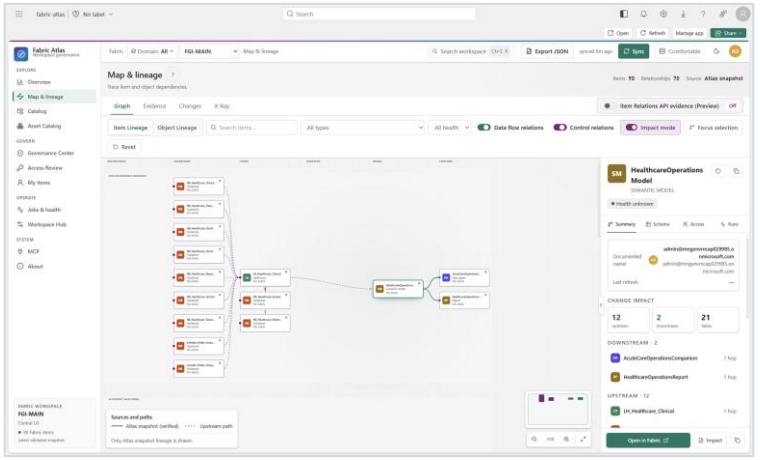
product 2.7.12 with live deployment evidence

Principles kept throughout: metadata only, validated snapshots, read-only AI, and unknown stays unknown.

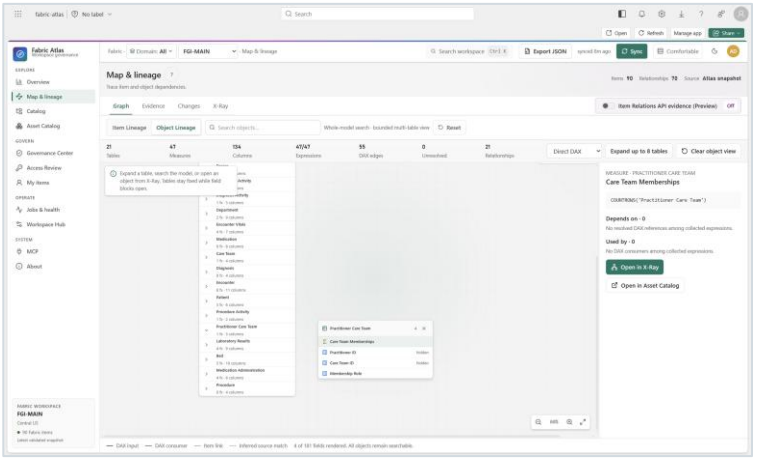
Six workspaces of evidence in one portal.



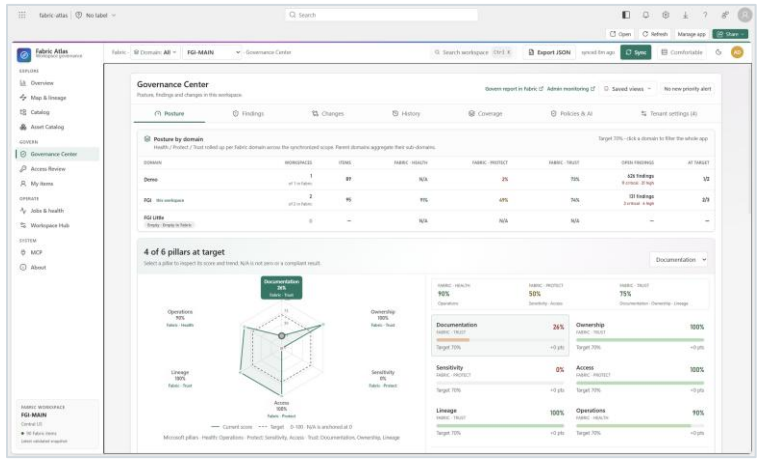
Catalog Health, owner, refresh and facets



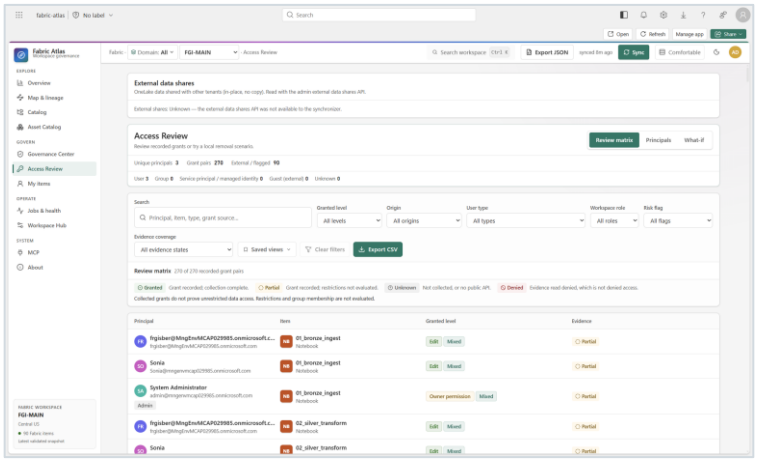
Map & lineage Trusted graph and impact analysis



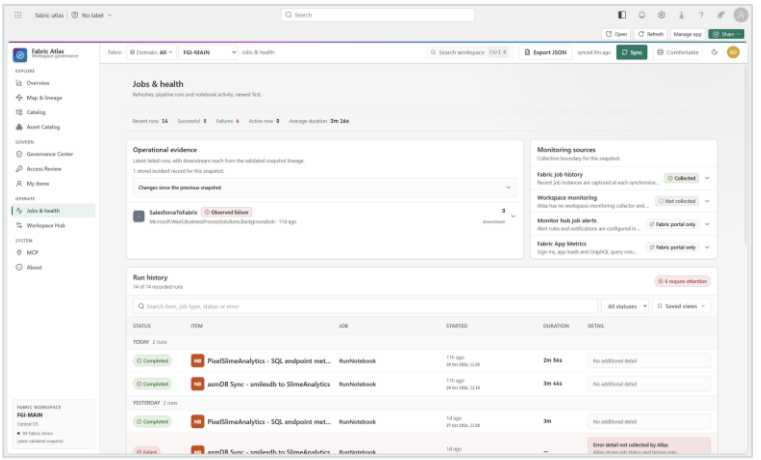
Object lineage Tables, columns, measures, DAX



Governance Center Findings, Why panel, Fix in Fabric



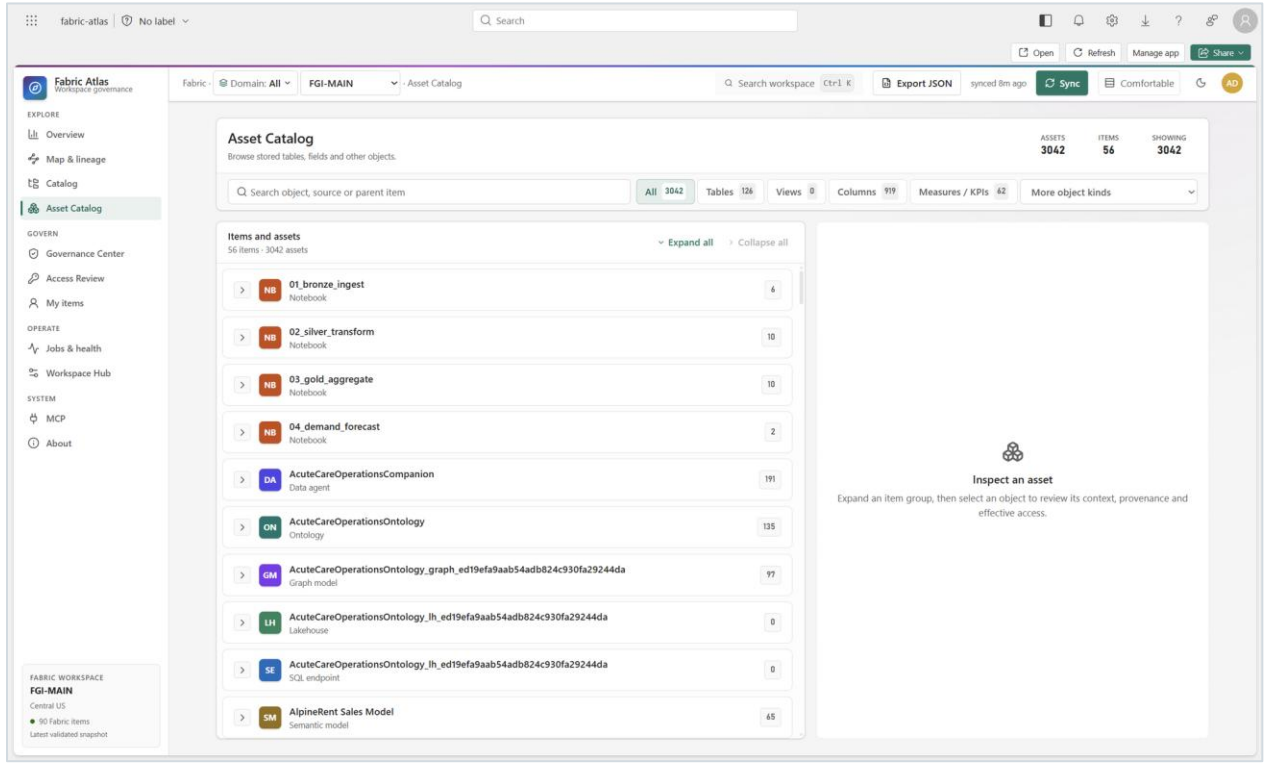
Access Review Effective access and decisions



Jobs & health Runs, failures and recoveries

MAJOR FEATURES

Governance outputs a team can act on.



Asset Catalog: the drawer hosts Ask Scout and Export identity sheet.

1

Ask Scout

2.7.11

One click from an item opens the Scout dock with a question about that item.

2

Identity sheet

2.7.11

Markdown one-pager: identity, ownership, trust, lineage, access, findings, recent jobs.

3

Sync Brief

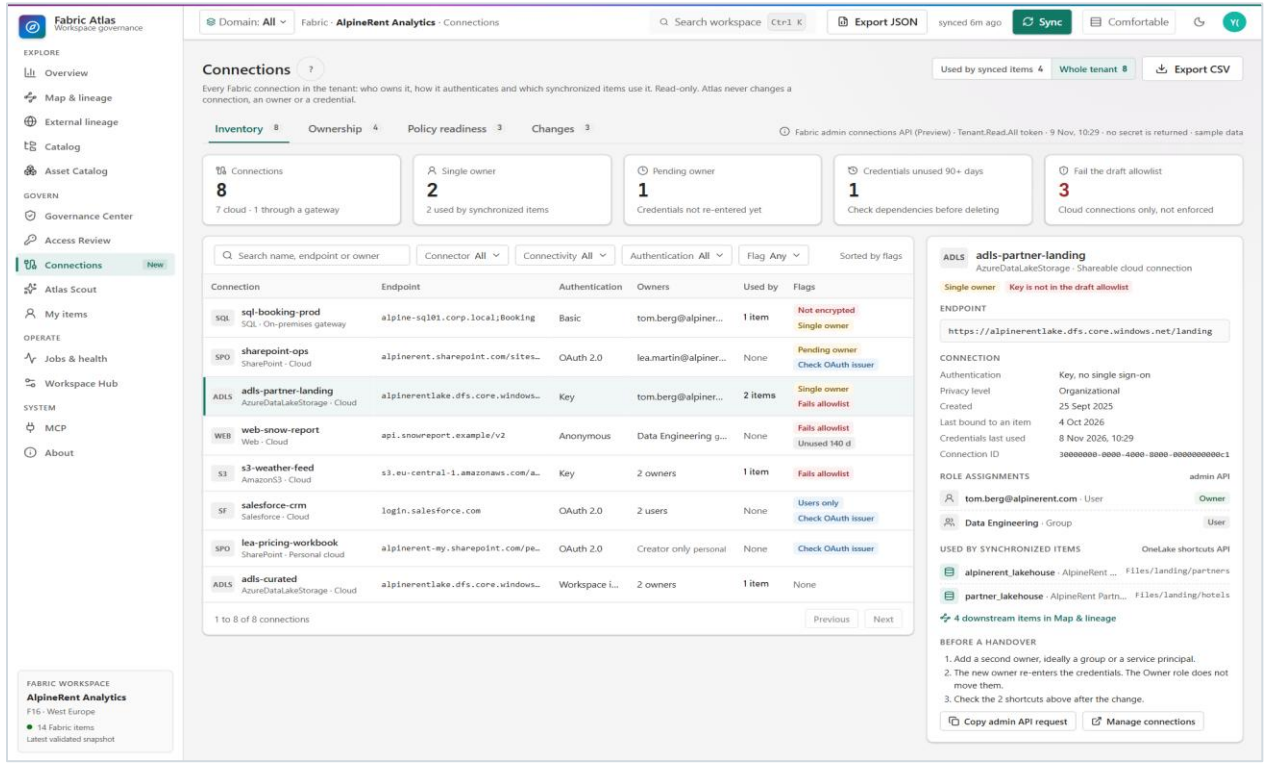
2.7.10

What changed since the last sync: items, external or elevated access, lineage, jobs, schema, new findings.

Read-only. Every output is built from the validated snapshot.

MAJOR FEATURES

Every Fabric connection, who owns it and what depends on it.



Govern → Connections, shown here on the built-in sample data.

1 Inventory — 2.8.0

Every connection of the tenant, its owners and the synchronized items that use it. Fabric returns no secret.

2 Ownership — 2.8–2.9

Single, pending or missing owner: a finding when synchronized items depend on it, and a check in every departure pack.

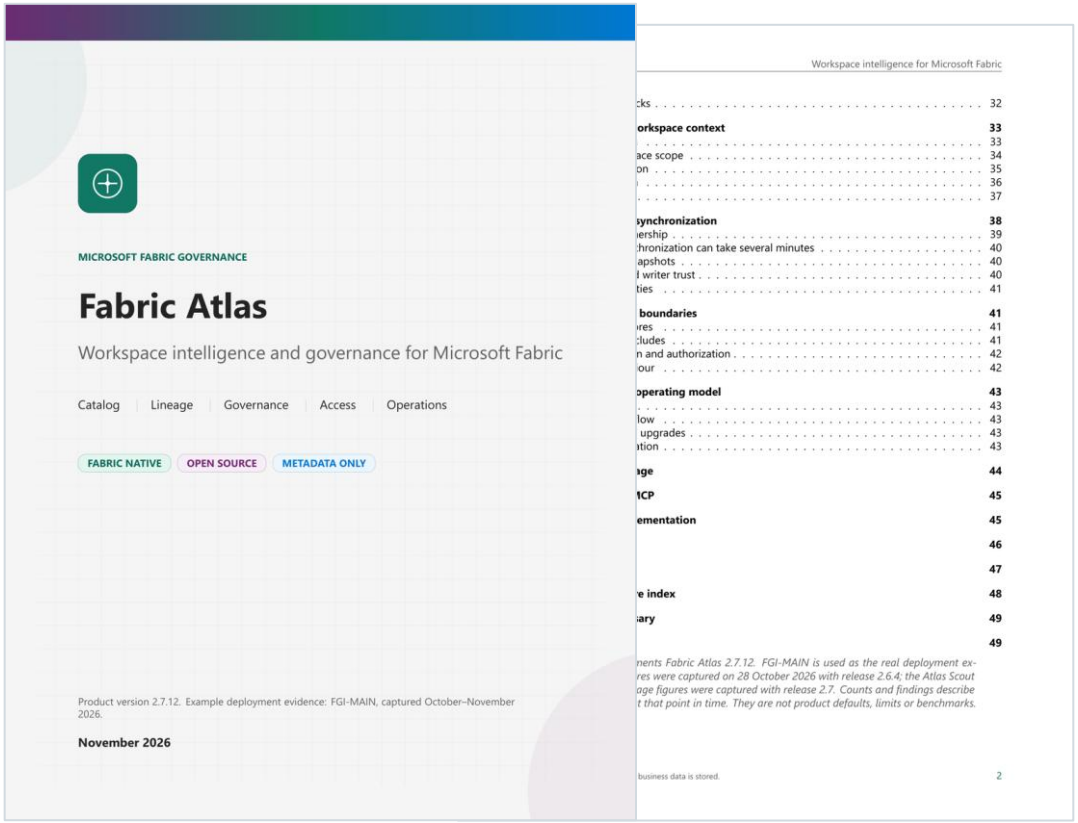
3 Policy readiness — 2.8.0

Test a draft authentication allowlist and blocked connection kinds before enforcing them in the admin portal.

Read-only. Atlas never changes a connection, an owner or a credential.

WHITEPAPER

The full product story, in one 50-page document.



MICROSOFT FABRIC GOVERNANCE

Fabric Atlas

Workspace intelligence and governance for Microsoft Fabric

Catalog | Lineage | Governance | Access | Operations

FABRIC NATIVE OPEN SOURCE METADATA ONLY

Product version 2.7.12. Example deployment evidence: FGI-MAIN, captured October–November 2026.

November 2026

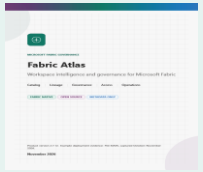
cks . . . 32
orkspace context 33
. . . 33
ace scope . . . 34
on . . . 35
. . . 36
. . . 37
synchronization 38
ership . . . 39
hronization can take several minutes . . . 40
apshots . . . 40
l writer trust . . . 40
ties . . . 41
boundaries 41
ires . . . 41
:ludes . . . 41
n and authorization . . . 42
our . . . 42
perating model 43
. . . 43
low . . . 43
upgrades . . . 43
ation . . . 43
ge 44
ICP 45
ementation 45
46
47
e index 48
ary 49
49

nents Fabric Atlas 2.7.12. FGI-MAIN is used as the real deployment ex-res were captured on 28 October 2026 with release 2.6.4; the Atlas Scout age figures were captured with release 2.7. Counts and findings describe t that point in time. They are not product defaults, limits or benchmarks.

Fabric Atlas whitepaper, product version 2.7.12

Workspace intelligence and governance for Microsoft Fabric

- Executive summary
- Workspace orientation
- Asset inventory
- Lineage and impact
- Governance and access
- Operations
- External lineage
- Atlas Scout



Embedded PDF

Double-click the icon to open the whitepaper. Online: github.com/fredgis/FabricAtlas, docs/fabric-atlas-whitepaper.pdf

AGENTIC TRANSFORMATION

From a governance catalog to an agentic governance analyst.

STEP 1 · v1.0 → 2.6

Evidence store

Atlas collects

- Validated snapshot per workspace, published manifest-last
- Object inventory: tables, columns, measures, DAX, lineage, OneLake roles
- Findings scored by Health, Protect and Trust

32 Rayfin entities

STEP 2 · v2.0 → 2.6.3

Agent-readable

Agents can read

- Local read-only Atlas MCP server for IDE and Copilot clients
- Catalog, lineage, access, connections, incidents, changes, findings, export
- Every answer carries snapshot, source, coverage and limitations

11 MCP evidence tools

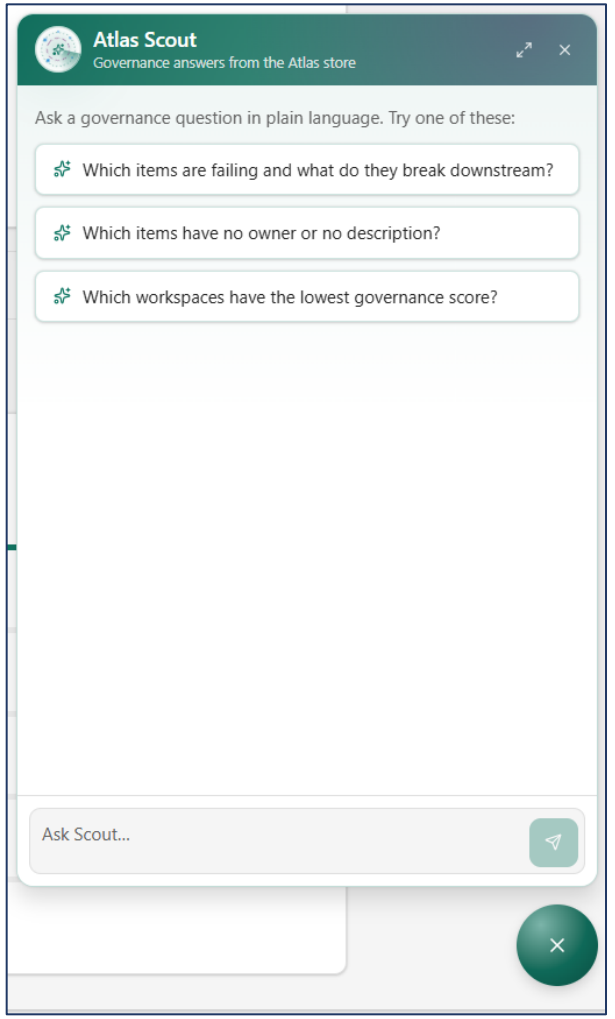
STEP 3 · v2.7.1 → 2.9.1

Agentic experience

Users ask

- Atlas Scout over a published Fabric Data Agent
- Page, floating bubble and Ask Scout share one conversation
- Answers as tables, gauges, status pills and lineage graphs

37 example queries



FOR THE USER

Before open the right tab, filter, read and cross-check the evidence yourself.

vNext ask in plain language and get the evidence, the impact and the next action.

AGENTIC TRANSFORMATION

Scout reasons like a governance analyst, inside strict guardrails.

METHOD

1 Audit
Query the latest snapshot

→

2 Explain
Say what the result means

→

3 Recommend
Next action, and where

EVERY FINDING HAS THE SAME SHAPE

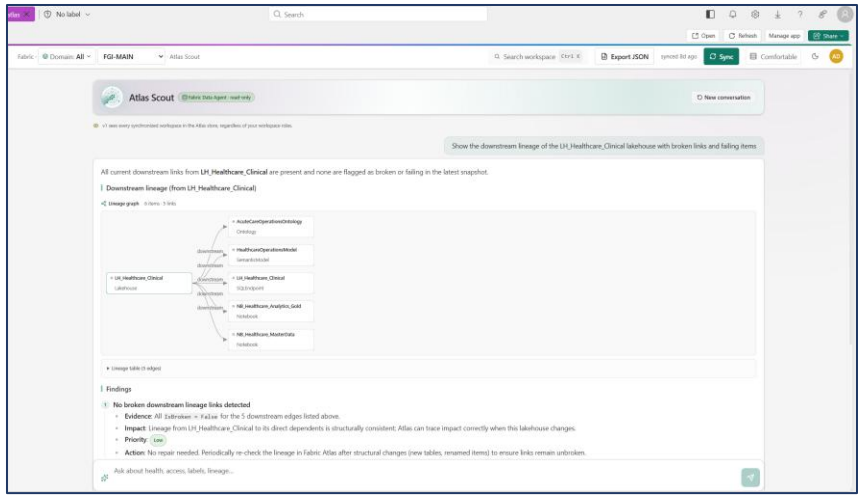
Finding	one sentence
Evidence	numbers and item names returned by the query, never invented
Impact	why it matters for the business or for compliance
Priority	Critical, High, Medium or Low
Action	what the owner should do, in Atlas Govern, OneLake catalog or workspace settings

GOVERNANCE PILLARS

Health
failures, broken lineage, stale items, no owner or domain

Protect
labels, guests, service principals, shares, admins

Trust
endorsement, descriptions, tags, ownership



GUARDRAILS

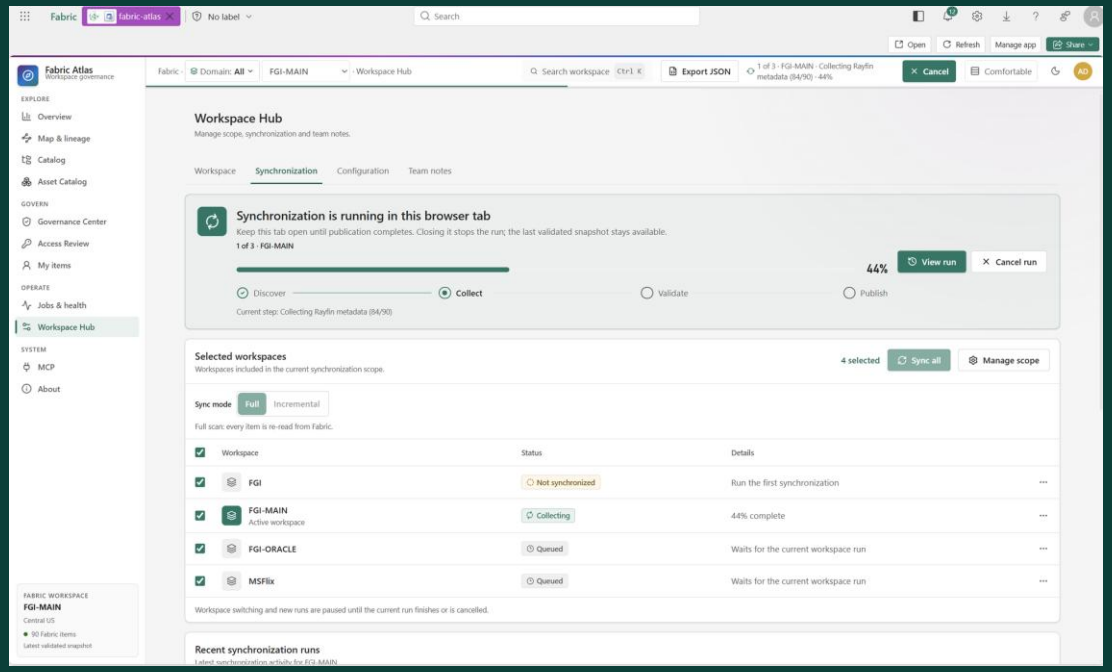
- **Read-only.** Never proposes INSERT, UPDATE or DELETE; humans apply changes.
- **Latest snapshot** of each workspace, unless a trend is asked.
- **Not measurable** is never counted as non-compliant; 0 % is not unknown.
- **Query first** before saying something is unavailable.
- **No view counts:** reach is the usage signal, and Scout says so.
- **User's language** for every answer.

v1 limit no row-level scope: users see all synced workspaces.

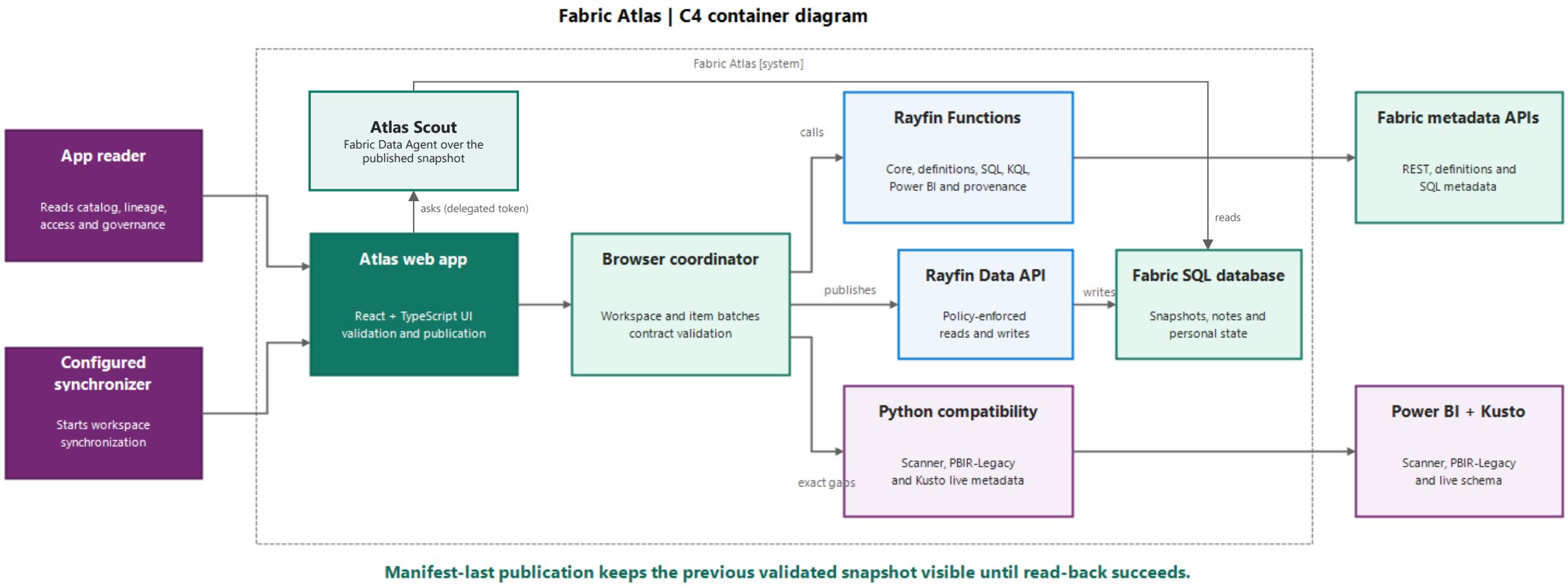
ARCHITECTURE AND IMPLEMENTATION

How Fabric Atlas is built

Rayfin Functions collect, Fabric SQL stores validated snapshots, the browser renders and Atlas Scout reads through a Fabric Data Agent.



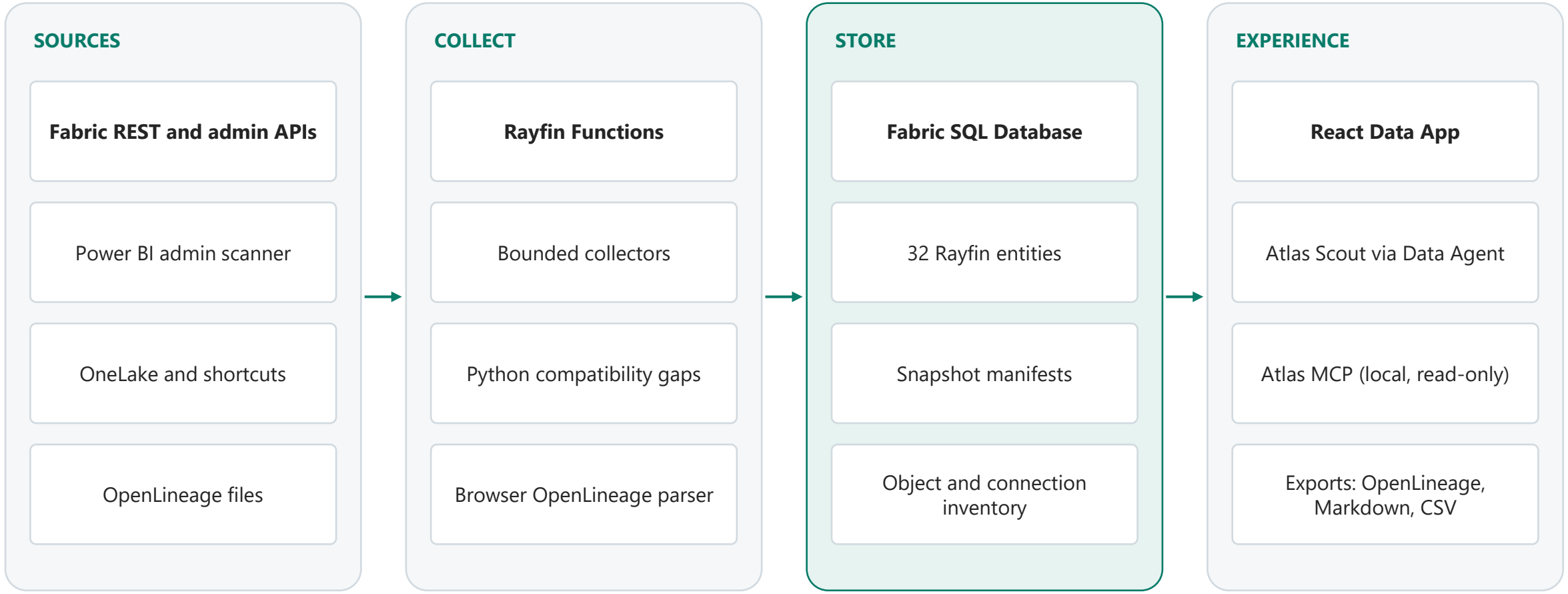
Rayfin Functions collect. The browser validates and publishes.



Python remains only for the Power BI scanner, PBIR-Legacy and Kusto gaps.

ARCHITECTURE 2.9.0

Four layers, one validated snapshot in the middle.



Rule: pages and Scout read the published snapshot. A failed sync never replaces the previous one.

Three platform gaps keep the Python compatibility UDF.

1. Power BI application audience or first-party scanner connector.

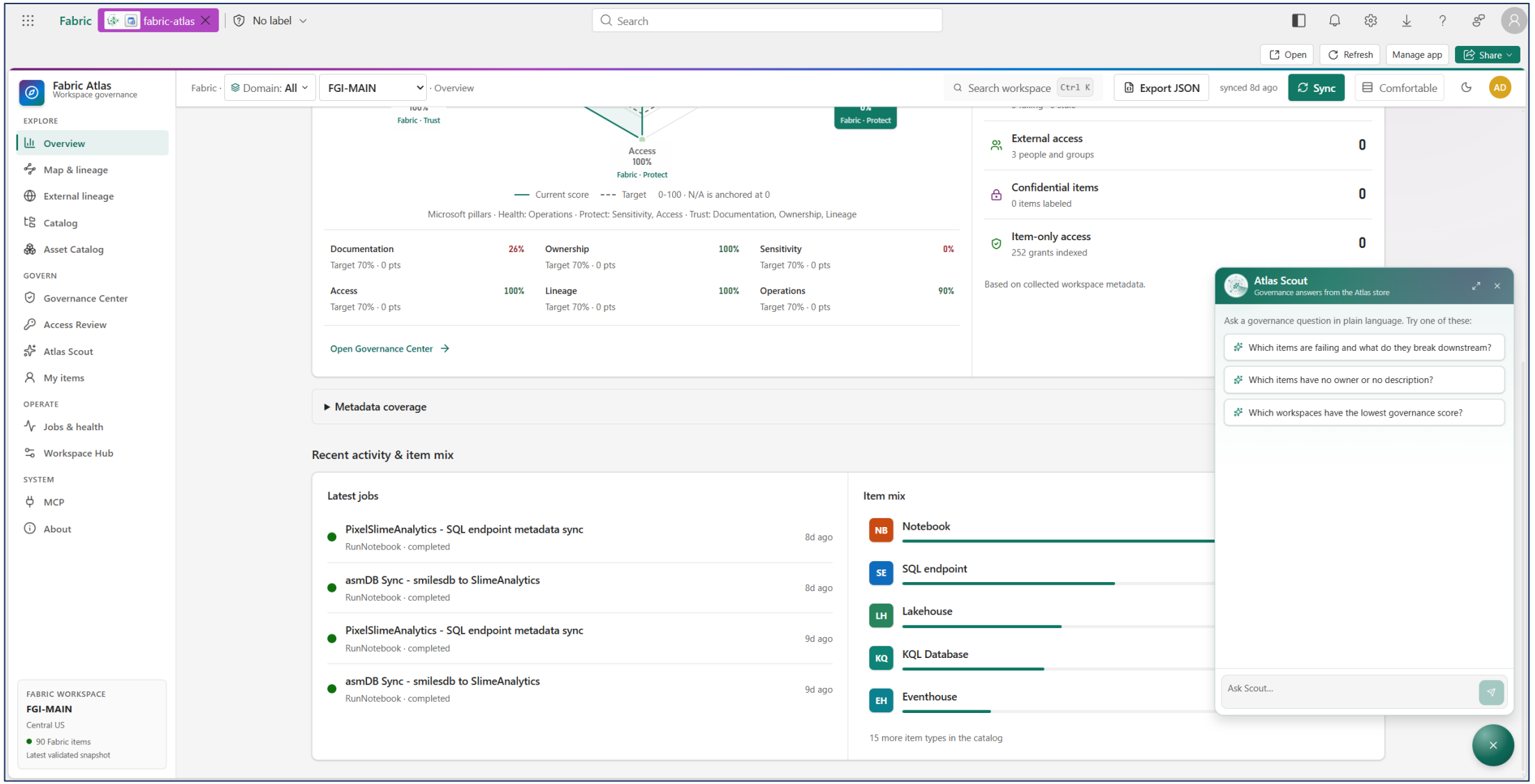
2. Kusto application audience for live schema metadata.

3. PBIR-Legacy report-page coverage.

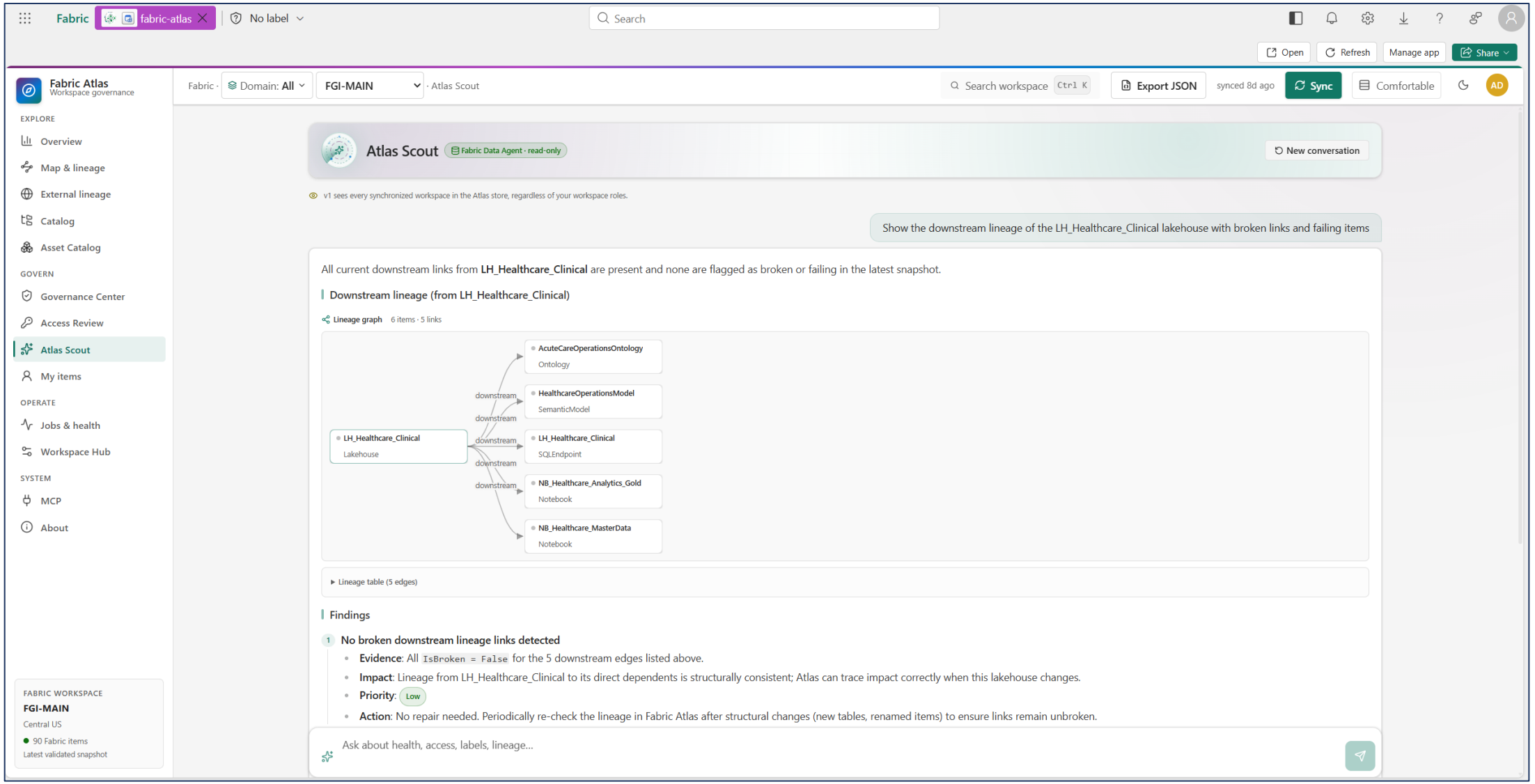
4. Once all three pass parity, delete sync_compatibility and the Python UDF.

Scheduling is separate: it needs an unattended trigger plus durable claim and publication fencing.

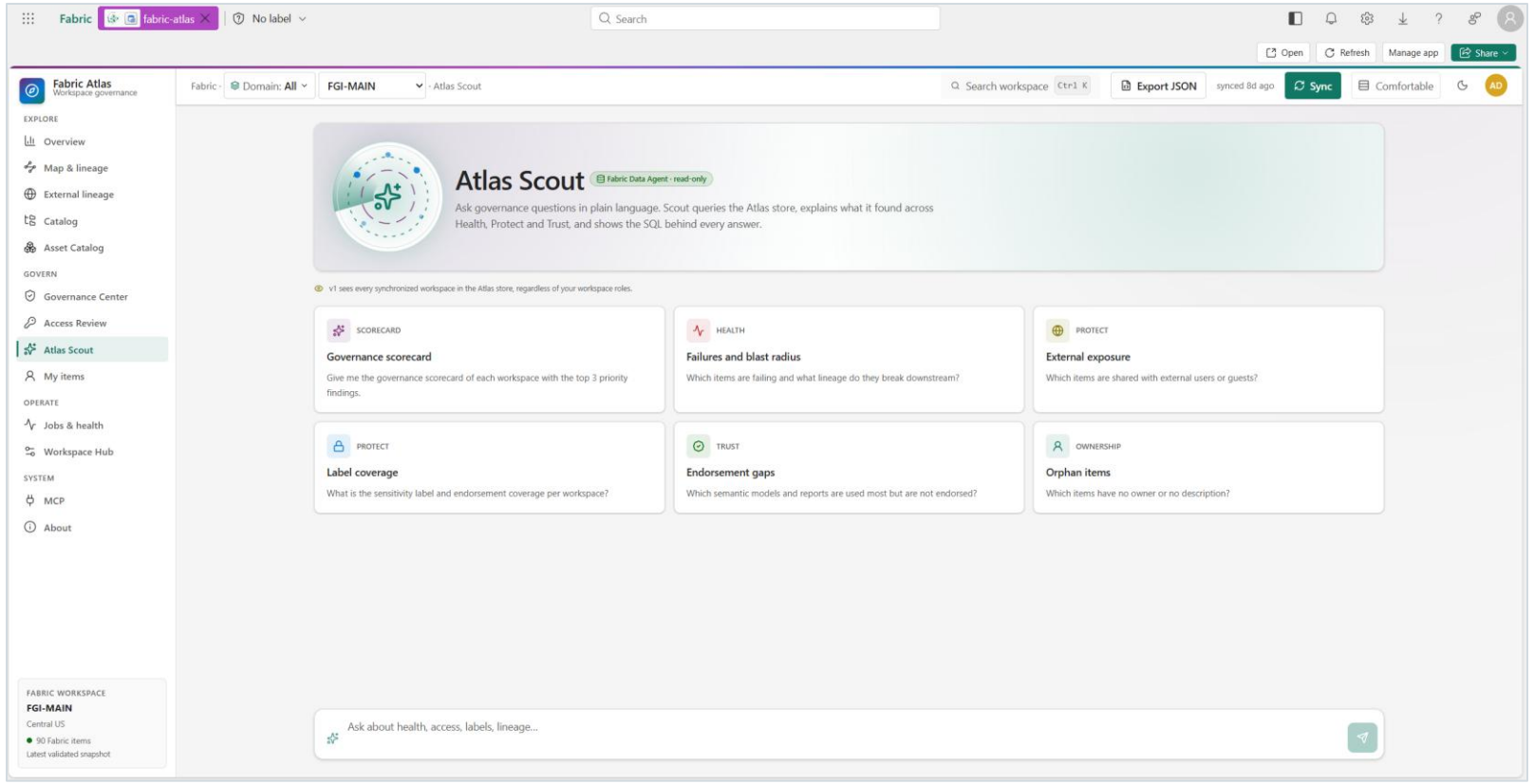
Atlas Scout stays available without taking over the workspace.



Atlas Scout draws lineage answers as a graph, then names the finding.



A full page with starter questions for each governance pillar.



What a user gets

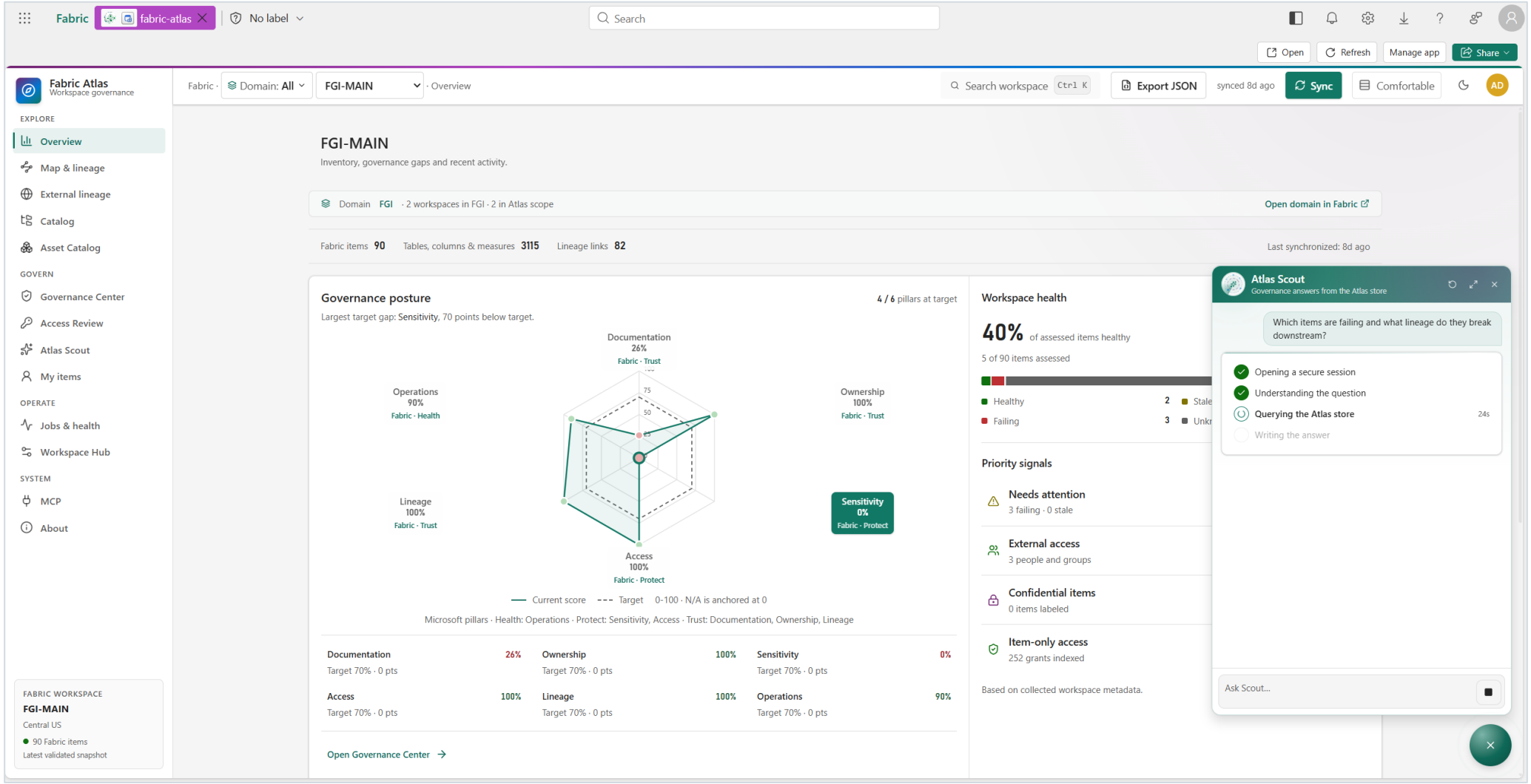
Starter questions for Health, Protect, Trust and Ownership

Tables, gauges, pills and numbered next actions

Evidence panel with the SQL steps and raw rows

Threads reused across follow-up questions

Scout shows its progress while the agent queries the store.



Atlas Scout calls a published Fabric Data Agent from the browser.

Atlas web app (browser)

Three entry points, one conversation

- **Govern → Atlas Scout page**
- **Floating Scout bubble**
- **Ask Scout (Catalog drawer)**

scout-chat.tsx shares the thread

scout.ts creates thread, posts, polls, Stop cancels

scout-markdown.ts tables, gauges, lineage graphs

DataAgent.Execute.All delegated

Fabric Data Agent

atlas-scout, published

Instructions with Health, Protect and Trust governance skills

37 example queries, including lineage up to four hops

Versioned in fabric/data-agent/atlas-scout with an idempotent provision.py

Returns the answer, the SQL steps and the result rows

read-only SQL

Atlas SQL database

Fabric SQL, Rayfin entities

Validated snapshots of every synchronized workspace

Object inventory: tables, columns, measures, DAX dependencies, relationships, domains, OneLake roles

Tenant connection inventory: owners, authentication, usage

Written only by Rayfin sync

Security. First use asks for consent to DataAgent.Execute.All. v1 has no row-level scope: every agent user sees every synchronized workspace. Optional runtime values: RAYFIN_PUBLIC_ATLAS_SCOUT_AGENT_ID and _WORKSPACE_ID.

Scout never writes to Atlas. The Data Agent reads only the Atlas SQL database.

SYNCHRONIZATION PROCESS

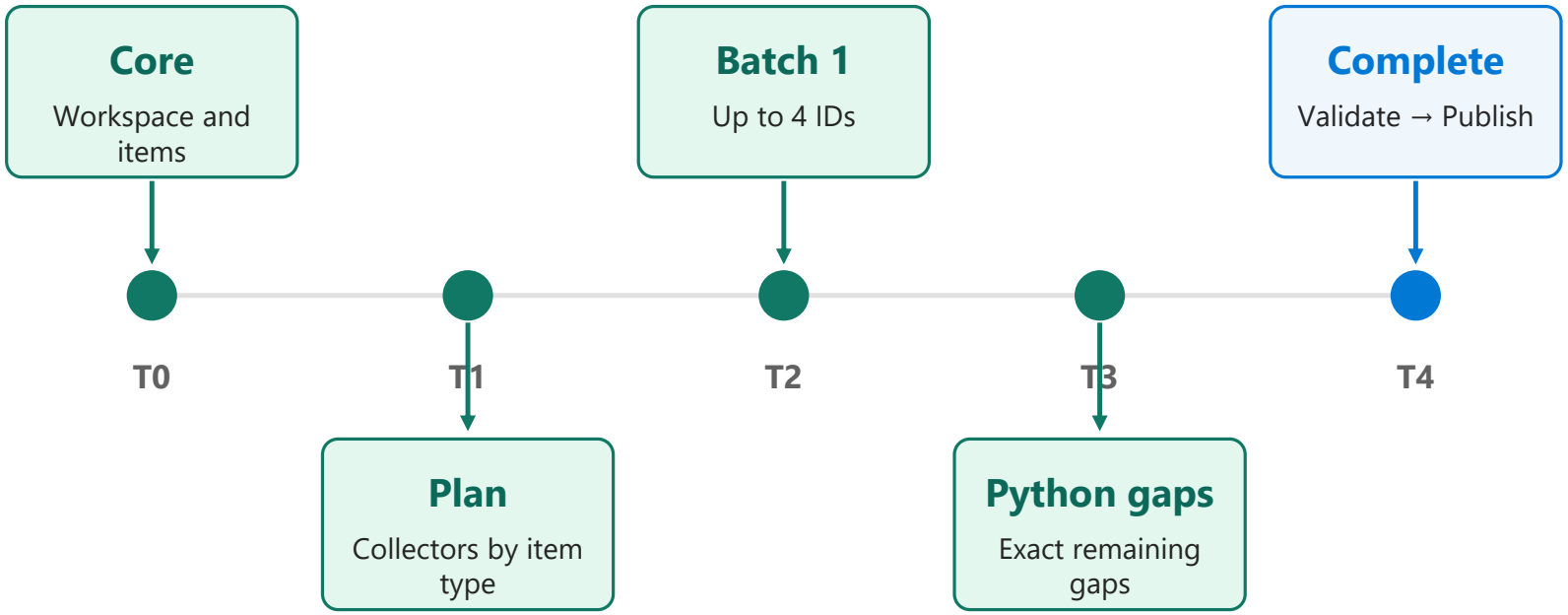
Synchronization follows five stages and publishes the manifest last.



The previous snapshot stays visible until the Workspace manifest is created.

BOUNDED COLLECTION

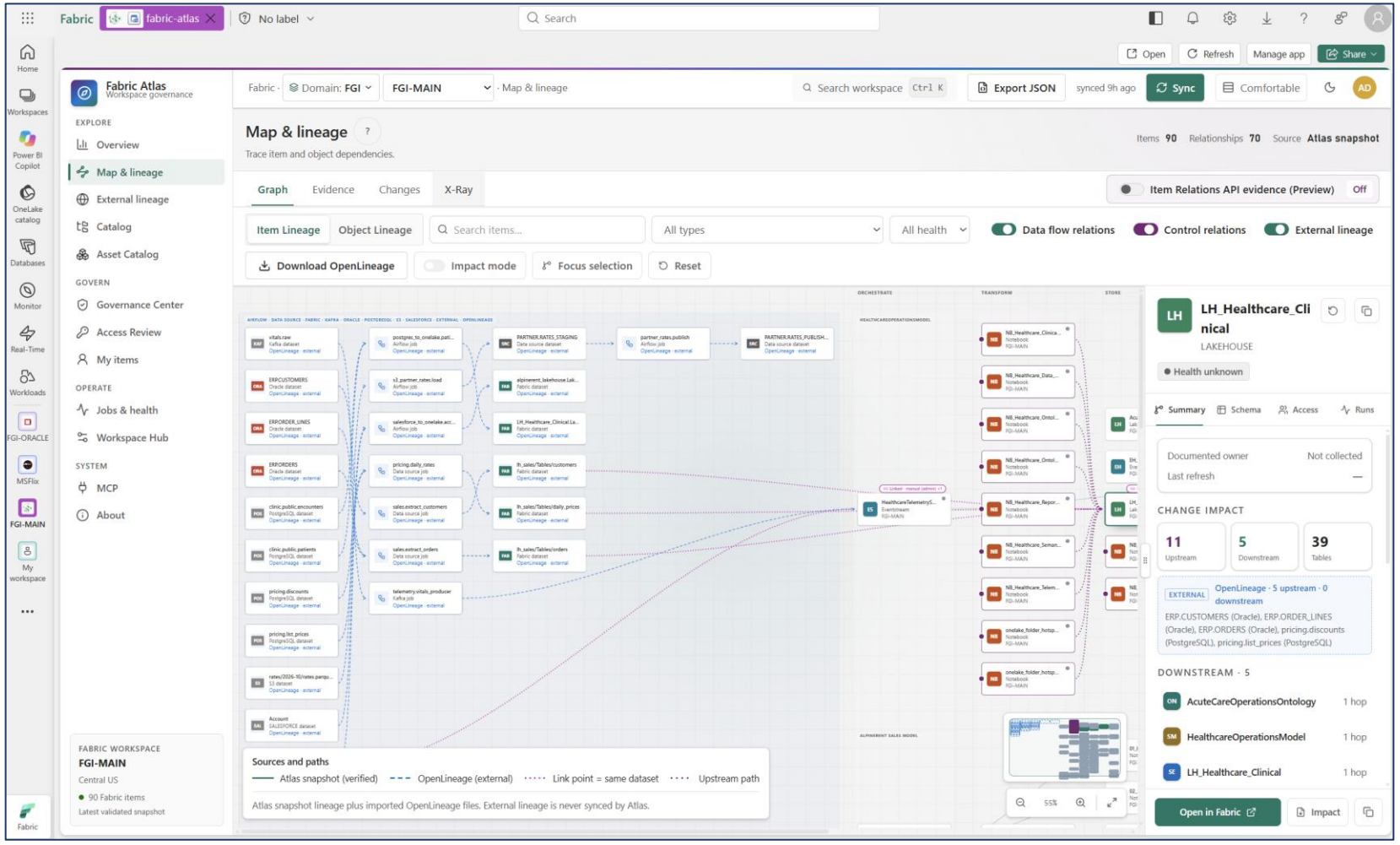
Rayfin Functions process large workspaces without restarting the run.



- Core creates the workspace and item plan.
- Rayfin collectors handle Ontology 2.0, Item Relations, KQL, SQL and Power BI.
- Python runs only the requested scanner, PBIR-Legacy and Kusto gaps.

Every invocation has bounded time, attempts, pages and response size.
Completed evidence is retained while remaining work continues.

Bring lineage from outside Fabric with OpenLineage.



Shipped in 2.7.0

1. Import an OpenLineage file (RunEvent, JobEvent, DatasetEvent). Parsing and validation run in the browser.

2. Only metadata is stored: OpenLineageImport, ExternalDataset and ExternalLineageEdge.

3. Exact OneLake paths link automatically; other datasets are linked by hand.

4. Map shows an External lane; Impact crosses into external consumers.

5. Export the Atlas lineage as typed OpenLineage NDJSON with the fabricAtlas_item facet.

OpenLineage works both ways: import external lineage, export Atlas lineage.

1 Import

.json or .ndjson

RunEvent, JobEvent, DatasetEvent

2 Validate

In the browser

20 MB, 50 000 events, 5 000 datasets

3 Store

OpenLineageImport

ExternalDataset

ExternalLineageEdge

4 Link

Exact OneLake path: auto

Others: manual, kept on re-import

5 Use

External lane on the Map

Impact across the boundary

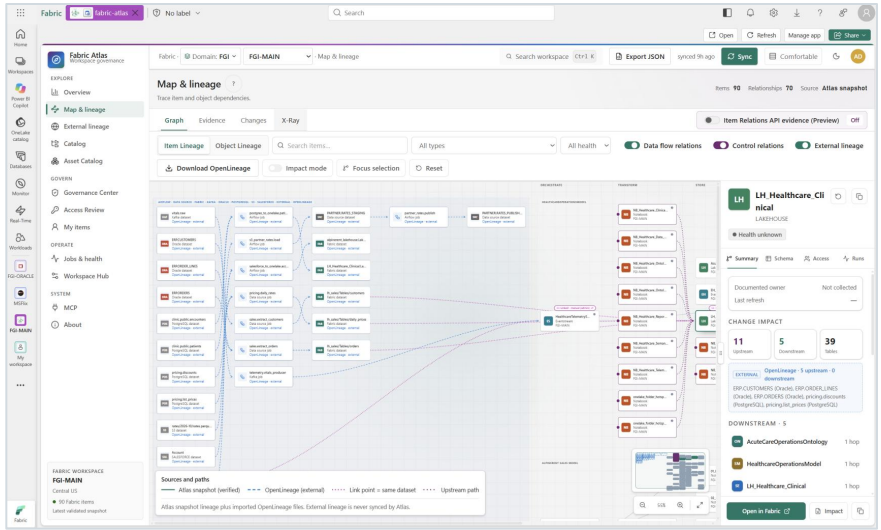
6 Export

NDJSON, abfss:// names

fabricAtlas_item facet

Exported DatasetEvent (excerpt)

```
{"eventType": "DatasetEvent",
 "dataset": {"namespace": "abfss://<workspace>@onelake.dfs...",
   "name": "LH_Sales/Tables/fact_sales",
   "facets": {"fabricAtlas_item": {
      "itemType": "Lakehouse", "workspaceId": "...",
      "_schemaURL": ".../FabricAtlasItemFacet.json"}}}}
```



Map with the External · OpenLineage lane

Rayfin receives thirty-two entity classes across four lifetimes.

Snapshot catalog

Workspace, FabricItem, LineageEdge, Principal, AccessGrant, JobRun and ConfigEntry

Shared control state

Comment, policies, exceptions, scope, OpenLineage, sync authority and connections

Sync and evidence state

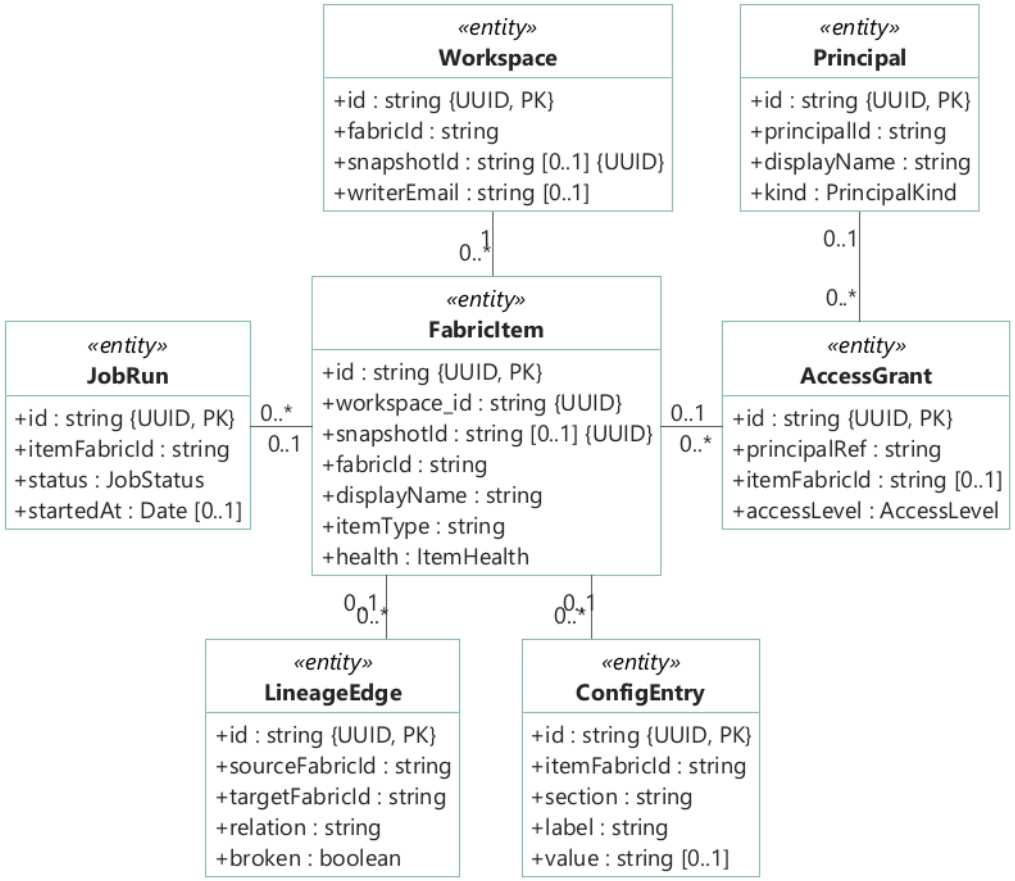
Sync runs, tasks, commands, payload records and evidence snapshots

Personal state

SavedView, AccessReview, AccessReviewEvent and FindingAck
are filtered by user_id.

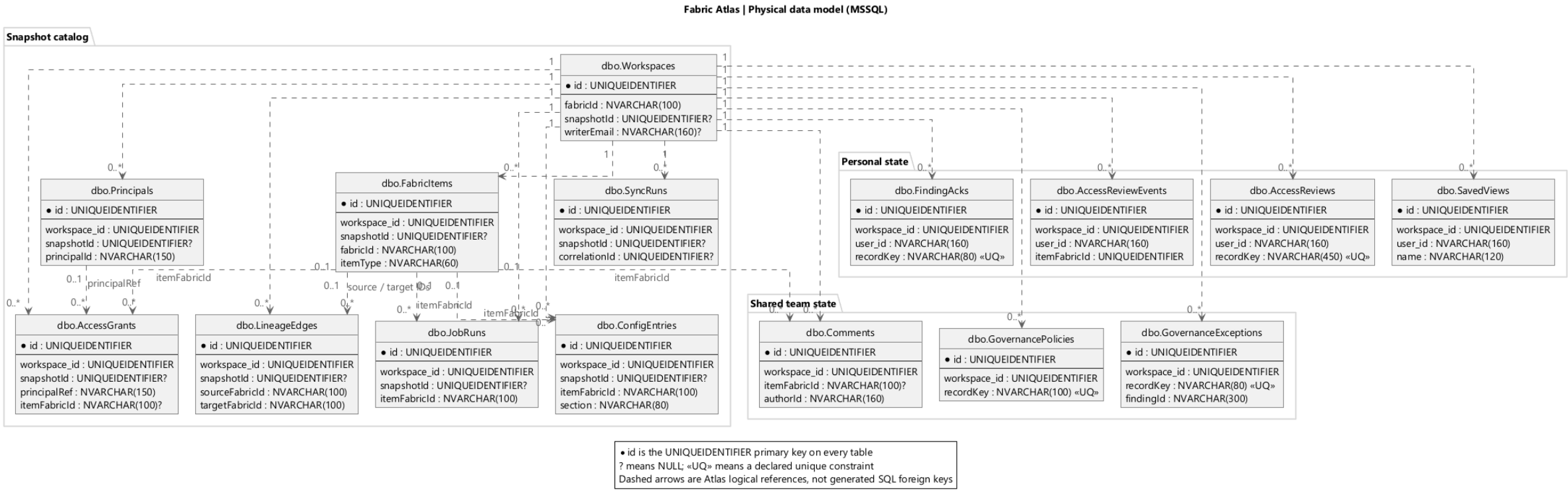
The entity classes define the database schema and the authorization boundary.

The core UML shows snapshot entities and logical associations.



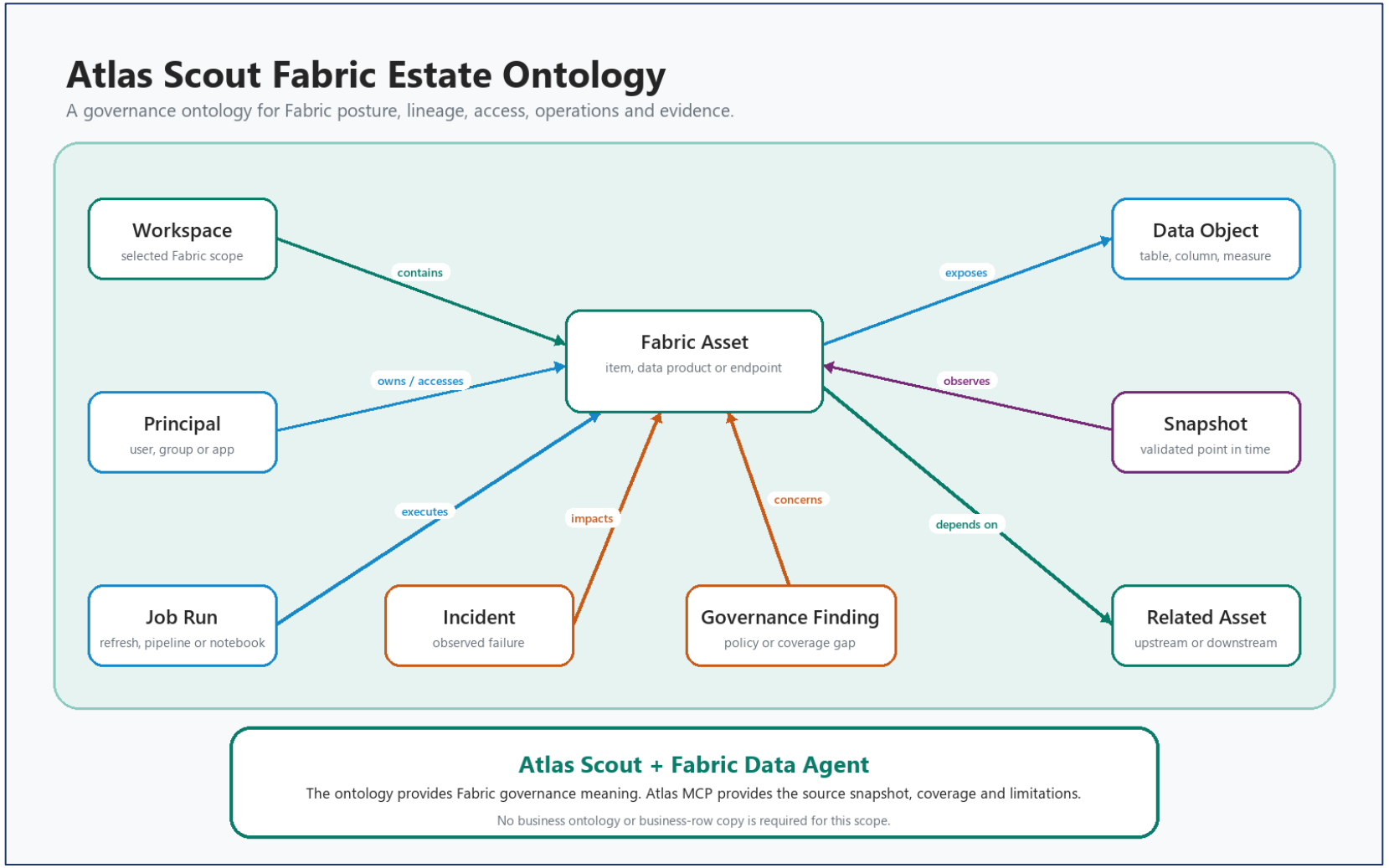
- Selected properties mirror the core TypeScript entities
- Workspace publishes a manifest; FabricItem anchors item evidence
- Additional v2 sync and evidence entities are grouped outside this core view.

The MPD shows the fifteen core MSSQL tables and logical links.



Fabric Atlas 2.9 keeps seventeen sync, scope, evidence, external lineage and connection tables outside this core view.

Map the Atlas data model into a Fabric Estate Ontology.



Proposed source path

1. Fabric SQL Database stays the system of record.

2. Project only the ontology entities and relationships into a Lakehouse.

3. Use OneLake shortcuts if supported. Otherwise use a bounded materialization.

4. Bind Fabric IQ Ontology to the Lakehouse. Atlas MCP keeps provenance, coverage and limitations.

DEMO



MICROSOFT FABRIC GOVERNANCE

Fabric Atlas

Workspace intelligence for Microsoft Fabric.

Catalog · lineage · governance · access · operations

OPEN SOURCE MIT LICENSED FABRIC NATIVE

FABRIC ATLAS

A trusted catalog stays available while the next snapshot is built.

Collect with Rayfin Functions. Keep exact compatibility gaps. Publish after read-back.

Questions and feedback

Fabric Atlas github.com/fredgis/FabricAtlas
Fabric Toolbox microsoft/fabric-toolbox · accelerators/fabric-atlas (PR #659)
Awesome Rayfin microsoft/awesome-rayfin · templates/fabric-atlas (PR #207)

Rayfin Functions · Fabric SQL · Multi-workspace · Metadata only